

Lab 11: Linux Essentials

Brittany Rose

CITC 1332 Linux Operating System

Instructor: Dr. Noman Saied

04/02/2026

Table of Contents

11.1 Introduction	3	11.2.12 Step 12	15
11.2	4	11.2.13 Step 13	16
11.2.1 Step 1	4	11.2.14 Step 14	17
11.2.2 Step 2	5	11.2.15 Step 15	18
11.2.3 Step 3	6	11.2.16 Step 16	19
11.2.4 Step 4	7	11.2.17 Step 17	20
11.2.5 Step 5	8	11.2.18 Step 18	21
11.2.6 Step 6	9	11.2.19 Step 19	22
11.2.7 Step 7	10	11.2.20 Step 20	23
11.2.8 Step 8	11	11.2.21 Step 21	24
11.2.9 Step 9	12	11.2.22 Step 22	25
11.2.10 Step 10	13	11.2.23 Step 23	26
11.2.11 Step 11	14	11.2.24 Step 24	27
11.2.25 Step 25	28	11.2.40 Step 40	43
11.2.26 Step 26	29	11.2.41 Step 41	44
11.2.27 Step 27	30	11.2.42 Step 42	45
11.2.28 Step 28	31	11.2.43 Step 43	46
11.2.29 Step 29	32	11.2.44 Step 44	47
11.2.30 Step 30	33	11.2.45 Step 45	48
11.2.31 Step 31	34	11.2.46 Step 46	49
11.2.32 Step 32	35	11.2.47 Step 47	50
11.2.33 Step 33	36	11.2.48 Step 48	51
11.2.34 Step 34	37	11.2.49 Step 49	52
11.2.35 Step 35	38	11.2.50 Step 50	53
11.2.36 Step 36	39	11.2.51 Step 51	54
11.2.37 Step 37	40	11.2.52 Step 52	55
11.2.38 Step 38	41	11.2.53 Step 53	56
11.2.39 Step 39	42	11.3.4 Step 4	67
11.2.54 Step 54	57	11.3.5 Step 5	68
11.2.55 Step 55	58	11.3.6 Step 6	69
11.2.56 Step 56	59	11.3.7 Step 7	70
11.2.57 Step 57	60	11.3.8 Step 8	71
11.2.58 Step 58	61	11.4	72
11.2.59 Step 59	62	11.4.1 Step 1	72
11.2.60 Step 60	63	11.4.2 Step 2	73
11.3	64	11.4.3 Step 3	74
11.3.1 Step 1	64	11.4.4 Step 4	75
11.3.2 Step 2	65	11.4.5 Step 5	78
11.3.3 Step 3	66		

11.1 Introduction

This is **Lab 11: Basic Scripting**. By performing this lab, students will learn how to use the vi editor to create basic shell scripts using basic shell commands, variables and control statements.

In this lab, you will perform the following tasks:

- Use the vi editor to create and edit text files.
- Create simple shell scripts.
- Create shell scripts with conditional execution.
- Use loops in the script for repetition.

11.2 Basic Text Editing

Most distributions of Linux have more than one text editor. These may include simple text-only editors, such as `nano`, or graphical editors, such as `gedit`.

In this task, you will explore some of the basic text editing features of the `vi` editor. All distributions have some version of `vi`. The `vi` editor is a powerful text editor with a bit of a learning curve, but capable of performing a wide variety of text editing tasks.

The `vi` editor has two modes: insert and command. In insert mode, you add text to a document. In command mode, operations such as navigation, searching, saving, and exiting the editor can be performed.

11.2.1 Step 1

To create a new file, execute the following command:

```
vi myfile
```

```
sysadmin@localhost:~$ vi myfile
```

Type an **i** to enter the "insert" mode of **vi** (more about this later). Then enter the following text:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

Then press the **Esc** key to leave insert mode. Type **:wq** to write the file to disk and quit.

Note

Each of the previous commands will be covered in greater detail later in this lab. The purpose of the previous step was to create a file to work with during this lab.

11.2.2 Step 2

Invoke the `vi` editor to modify the file you created. When `vi` is invoked, you are placed in *command* mode by default:

```
vi myfile
```

Your output should be similar to the following:

```
Welcome to the vi editor.
It is a very powerful text editor.
Especially for those who master it.
```

```
"myfile" [New File]
```

11.2.3 Step 3

Press each of the following keys two times and observe how the cursor moves. Remember that you are in command mode:

Key	Function
j	Moves cursor down one line (same as down arrow)
k	Moves cursor up line (same as up arrow)
l	Moves cursor to the right one character (same as right arrow)
h	Moves cursor to the left one character (same as left arrow)
w	Moves cursor to beginning of next word
e	Moves cursor to end of word
b	Moves cursor to beginning of previous word

Warning: If you type any other keys than those listed above, you may end up in insert mode. Don't panic! Press the **Esc** key, then `:q!` + the **Enter** key. This should exit `vi` without saving any changes. Then execute `vi myfile` and you are back in the `vi` editor!

11.2.4 Step 4

More vi cursor navigation: press the following keys and observe how the cursor moves:

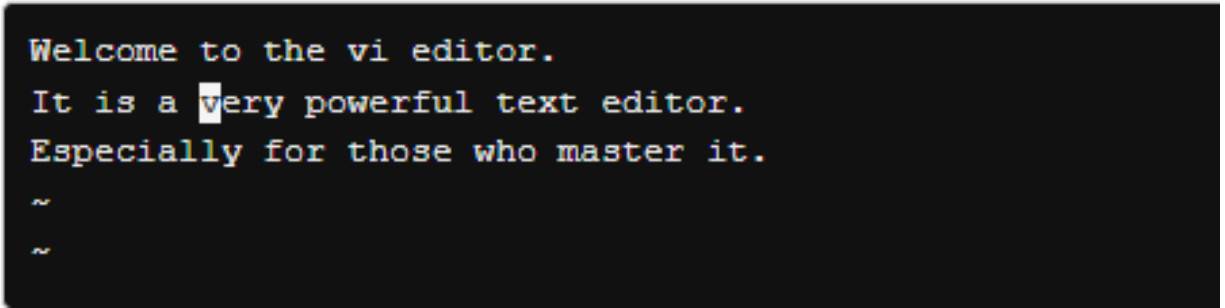
Keys	Function
<code>\$</code>	Moves cursor to end of current line (same as End key)
<code>0</code> (zero)	Moves cursor beginning of current line (same as Home key)
<code>3G</code>	Jumps to third line (<code>nG</code> jumps to the nth line)
<code>1G</code>	Jumps to first line
Shift+G	Jumps to the last line

11.2.5 Step 5

Move the cursor to the beginning of the word "very" by pressing the following keys:

```
3G (hold down the number "3" key and press the letter "g")  
k  
8l (that's the number eight followed by the letter "l")
```

The cursor should be on the letter v of the word "very" as shown below:



```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.  
~  
~
```

11.2.6 Step 6

Delete the word “very” by issuing the command `dw` (**d**ele**te** **w**ord):

 dw

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a powerful text editor.  
Especially for those who master it.
```

```
"myfile" [New File]
```

11.2.7 Step 7

Undo the last operation:

u

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

```
1 change; before #2 16:38:03
```

11.2.8 Step 8

Delete two words:

 $2dw$

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a text editor.  
Especially for those who master it.
```

```
1 change; before #2 16:38:03
```

11.2.9 Step 9

Undo the last operation:

u

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

1 change; before #3 91 seconds ago

11.2.10 Step 10

Delete four characters, one at a time:

XXXX

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a powerful text editor.  
Especially for those who master it.
```

1 change; before #3 91 seconds ago

11.2.11 Step 11

Undo the last 4 operations and recover the deleted characters:

 $4u$

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

~~~~~

4 changes; before #4 28 seconds ago

## 11.2.12 Step 12

Delete 14 characters:

14x

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a text editor.  
Especially for those who master it.
```

4 changes; before #4 28 seconds ago



### 11.2.13 Step 13

Undo the last operation:

u

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

~~~~~

1 change; before #8 65 seconds ago

11.2.14 Step 14

Delete the five characters to the left of the cursor (type 5 then **Shift+x**):

5X

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It very powerful text editor.  
Especially for those who master it.
```

1 change; before #9 9 seconds ago

11.2.15 Step 15

Undo the last operation:

u

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

~~~~~

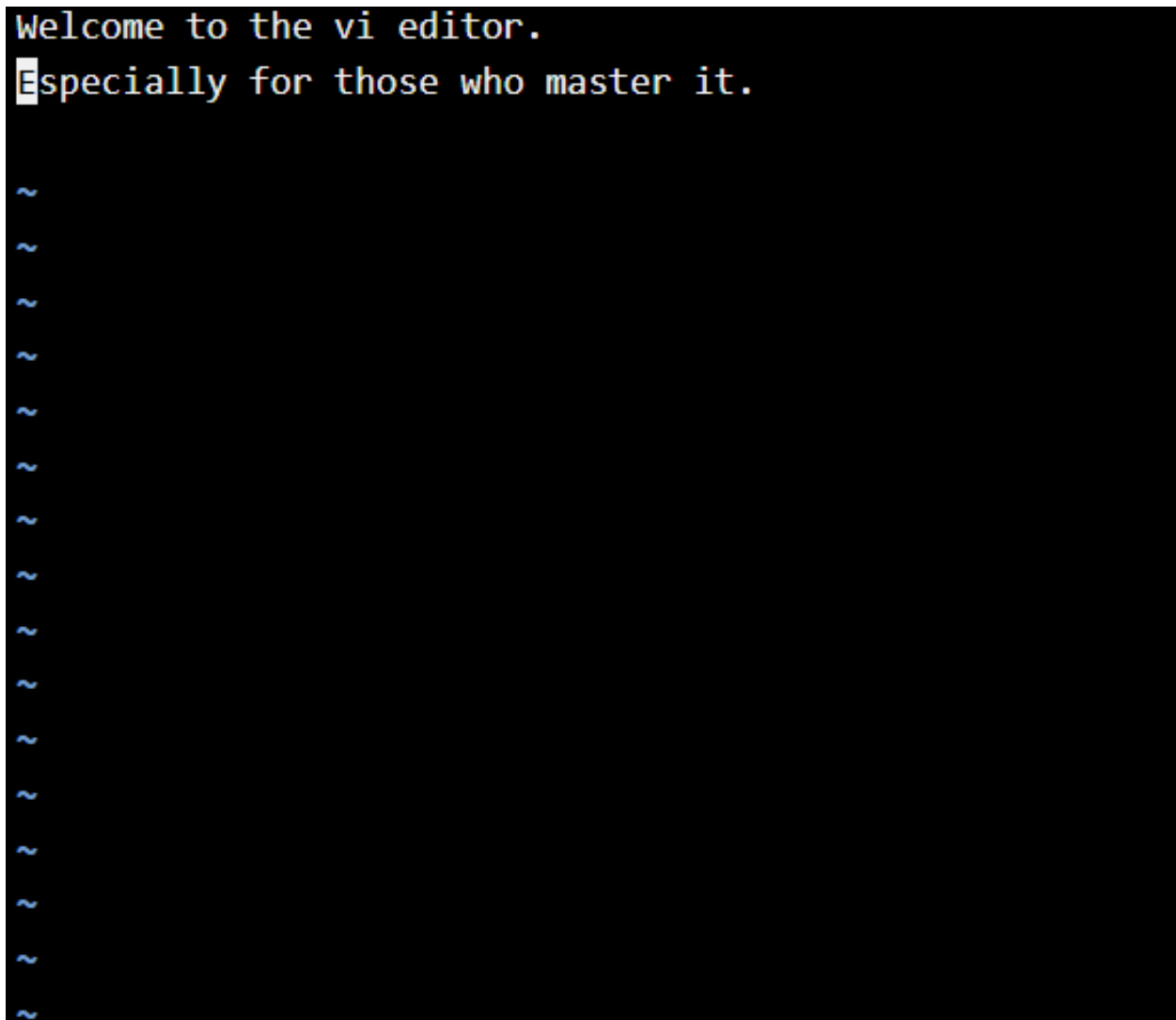
1 change; before #10 64 seconds ago

## 11.2.16 Step 16

Delete the current line:

```
dd
```

Your screen should look similar to the following:



The screenshot shows the vi editor interface on a black background. The first line contains the text "Welcome to the vi editor." and the second line contains "Especially for those who master it." The cursor is positioned at the start of the second line, indicated by a white block. Below these two lines, there are 15 lines of tilde (~) characters, representing the rest of the file. The text is displayed in a monospaced font with a light blue color.

## 11.2.17 Step 17

Whatever was last deleted or yanked can be “pasted”. Paste the deleted lines below the current line:

p

Your screen should look similar to the following:

```
Welcome to the vi editor.  
Especially for those who master it.  
It is a very powerful text editor.
```

~~~~~

11.2.18 Step 18

Undo the last two operations:

 $2u$

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

```
1 change; before #11 17:27:32
```

11.2.19 Step 19

Delete two lines, the current and the next:

2dd

Your screen should look similar to the following:

```
Welcome to the vi editor.
```



3

2

2

3

11.2.20 Step 20

Undo the last operation:

```
u
```

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```


11.2.21 Step 21

Move to the fourth word then delete from the current position to the end of the line **Shift+D**:

```
4w
```

```
D
```

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

The command **d\$** also deletes to end of line. The **\$** character, as seen earlier, advances to end of line. Thus, **d\$** deletes to end of line.

11.2.22 Step 22

Undo the last operation:

u

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

1 change; before #14 95 seconds ago

11.2.23 Step 23

Join two lines, the current and the next by typing a capital J (**Shift+J**):

J

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor. Especially for those who master it.
```

11.2.24 Step 24

Undo the last operation:

u

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

11.2.25 Step 25

Copy (or “yank”) the current word:

```
yw
```

When you copy text, no change will take place on the screen.

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

11.2.26 Step 26

Paste (or “put”) the copied word before the current cursor by typing **Shift+p**:

P

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful powerful text editor.  
Especially for those who master it.
```

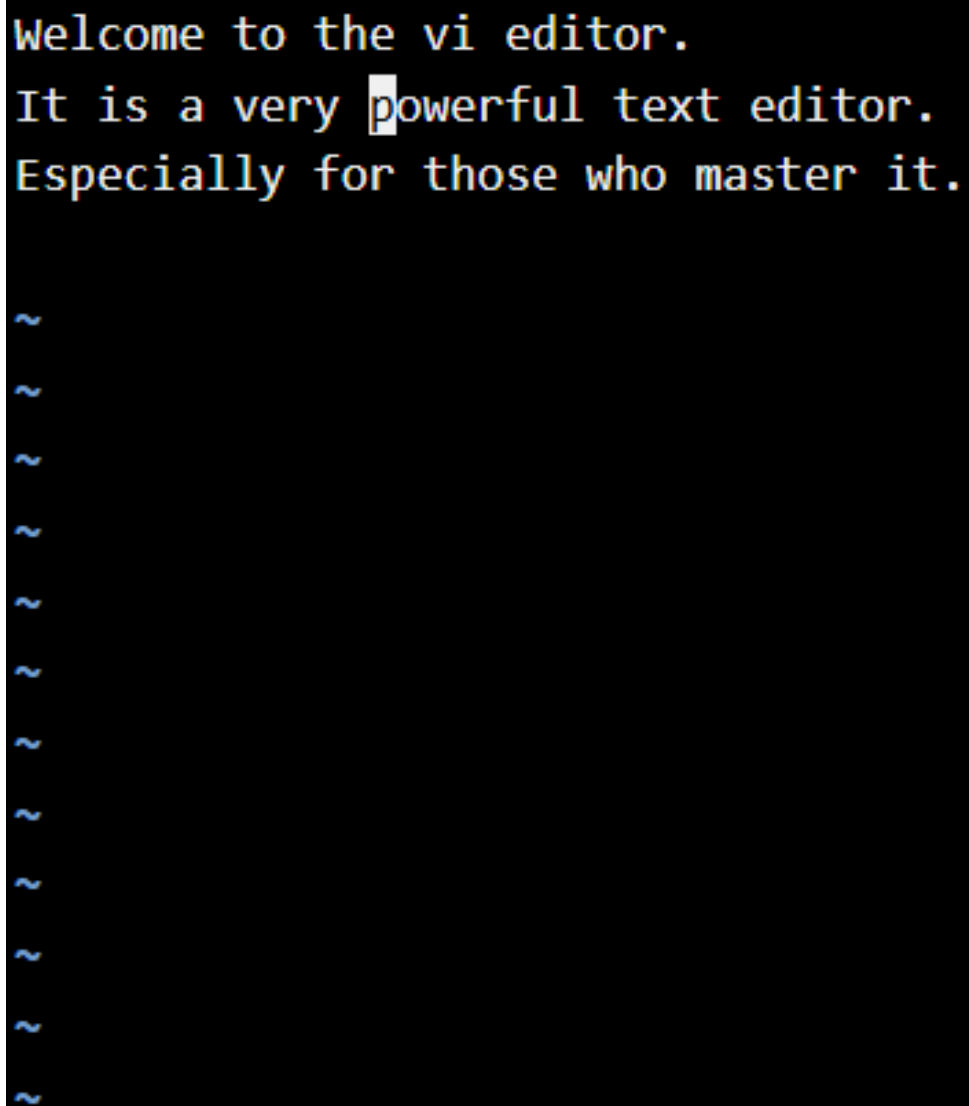
```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

11.2.27 Step 27

Undo the last operation:

```
u
```

Your screen should look similar to the following:



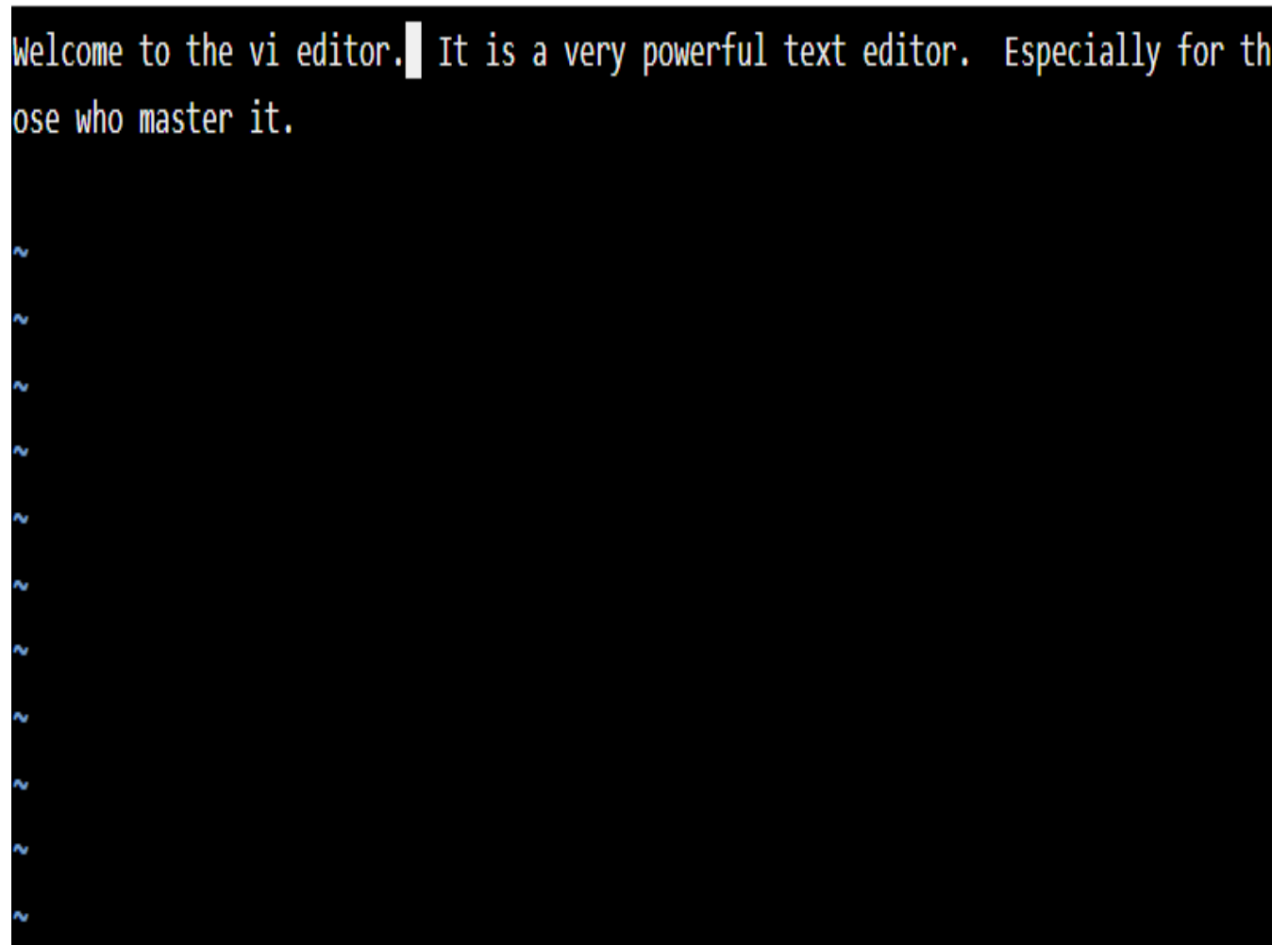
```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.  
  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

11.2.28 Step 28

Move to the first line, then join three lines:

```
1G  
3J
```

Your screen should look similar to the following:



```
Welcome to the vi editor. | It is a very powerful text editor. Especially for those who master it.  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```


11.2.29 Step 29

Undo the last operation:

u

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```



11.2.30 Step 30

Search for and delete the word `text` (add a space after the word `text`):

```
:%s/text //g
```

Your screen should look similar to the following:

```
Welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

~~~~~

```
:%s/text //g
```

## 11.2.31 Step 31

Navigate to beginning of file, then press `i` to enter insert mode to add text:

| Keys            | Function                                      |
|-----------------|-----------------------------------------------|
| <code>1G</code> | Go to beginning of file ( <b>Shift+G</b> )    |
| <code>i</code>  | Enter insert mode                             |
| Hello and       | Add text to document with a space after “and” |

```
Hello and Welcome to the vi editor.  
It is a very powerful editor.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

### 11.2.32 Step 32

Exit insert mode and return to command mode by pressing the **Escape** key:

ESC

```
Hello and Welcome to the vi editor.  
It is a very powerful editor.  
Especially for those who master it.
```

## 11.2.33 Step 33

Move forward one space by pressing the lower case `l` to place the cursor on the `w` and toggle it to lower case by pressing the tilde (`~`):

| Keys           | Function                                    |
|----------------|---------------------------------------------|
| <code>l</code> | Lowercase 'L' moves forward one space       |
| <code>~</code> | <b>Shift+`</b> changes letter to lower case |

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.
It is a very powerful editor.
Especially for those who master it.

~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
```

## 11.2.34 Step 34

Save the file. Press the **Esc** key to ensure you are in command mode. Then type `:w` and the **Enter** key:

$$\cdot W$$

```
Hello and welcome to the vi editor.  
It is a very powerful editor.  
Especially for those who master it.
```

```
"myfile" [New File] 4 lines, 103 characters written
```

## 11.2.35 Step 35

When you press **Enter** to commit the change, note the message in lower left indicating the file has been written:

[illegible]

## 11.2.36 Step 36

Navigate to the space between the word "powerful" and "editor" in the second line as shown in the image below. You could press j followed by 10l or use the arrow keys:

| Command | Function/Keys                    |
|---------|----------------------------------|
| j       | Move down to second line         |
| 10l     | 10 followed by the lowercase 'L' |

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful editor.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```



### 11.2.37 Step 37

Append text to the right of the cursor by pressing the letter a. This moves the cursor to the right and enters insert mode. Type the word text followed by a space as shown in image below:

| Command | Function/Keys            |
|---------|--------------------------|
| a       | Enter insert mode.       |
| text    | text followed by a space |

[illegible]

### 11.2.38 Step 38

Exit insert mode by pressing the **Esc** key.

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

```
"myfile" [New File] 4 lines, 103 characters written
```

## 11.2.39 Step 39

Open a blank line below the current line by typing a lowercase letter `o`:

`o`

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
|  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.40 Step 40

Enter the following text:

```
This line was added by pressing lowercase o.
```

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.41 Step 41

Exit insert mode by pressing the **Esc** key.

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.42 Step 42

Open a blank line above the current line by pressing uppercase **O**:

O

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
█  
This line was added by pressing lowercase o.  
Especially for those who master it.  
  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.43 Step 43

Enter the following text:

```
You just pressed O to open a line above.
```

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.█  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.44 Step 44

Exit insert mode by pressing the **Esc** key.

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```



## 11.2.45 Step 45

Save the file and close the `vi` editor using any one of the following methods that *saves* changes:

| Command           | Function/Keys                                                                  |
|-------------------|--------------------------------------------------------------------------------|
| <code>:x</code>   | Will save and close the file.                                                  |
| <code>:wq</code>  | Will write to file and quit.                                                   |
| <code>:wq!</code> | Will write to a read-only file, if possible, and quit.                         |
| <code>ZZ</code>   | Will save and close. Notice that no colon <code>:</code> is used in this case. |
| <code>:q!</code>  | Exit without saving changes                                                    |
| <code>:e!</code>  | Discard changes and reload file                                                |
| <code>:w!</code>  | Write to read-only, if possible.                                               |

## 11.2.46 Step 46

Once again open `myfile` using the `vi` editor:

```
vi myfile
```

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

```
"myfile" 6 lines, 194 characters
```

## 11.2.47 Step 47

Navigate to the third line, and delete the third and fourth lines:

```
3G  
2dd
```

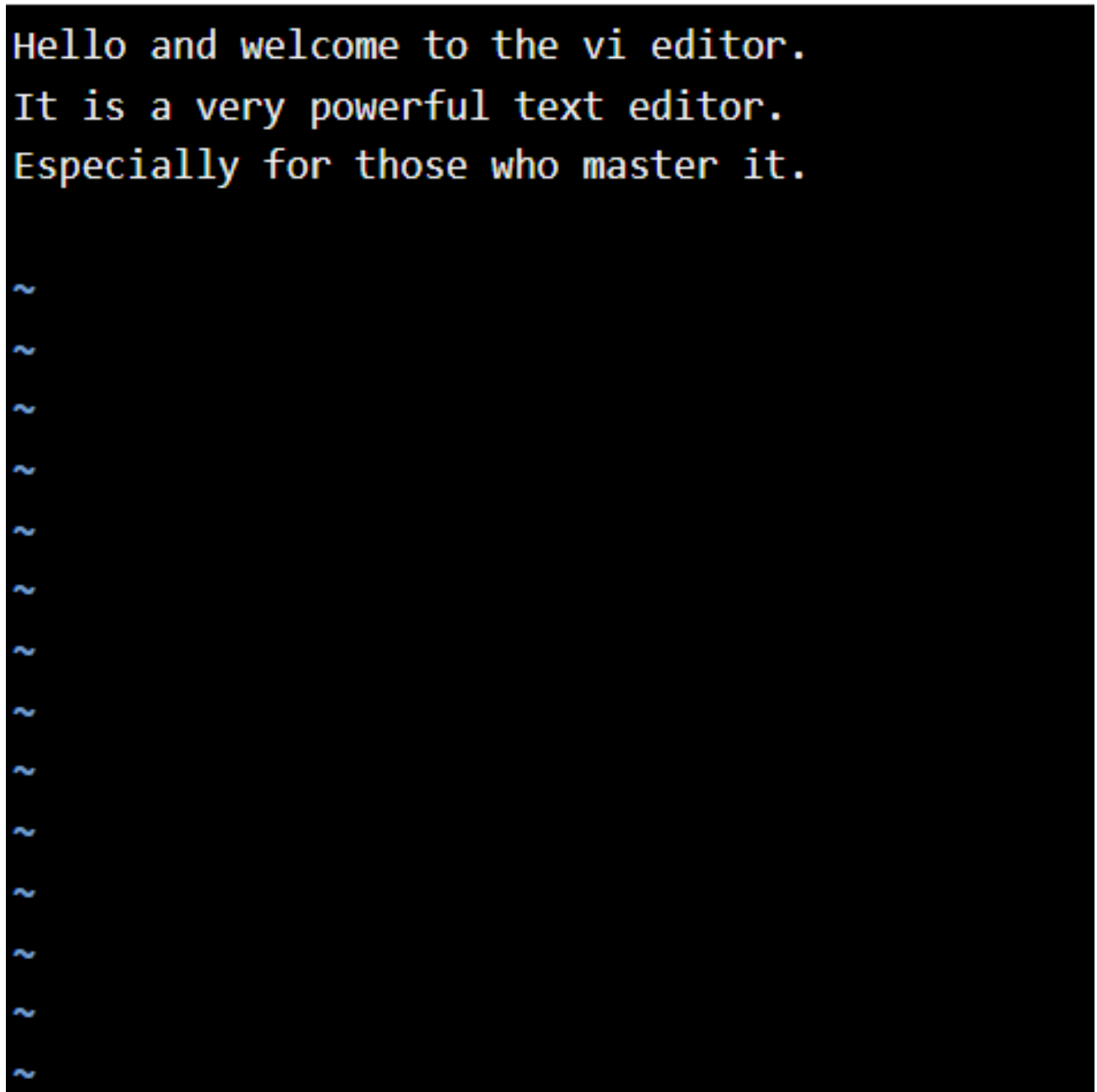
Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.48 Step 48

Press the **Esc** key to confirm you are in command mode.



```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
Especially for those who master it.  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.49 Step 49

Quit the `vi` editor without saving your changes:

```
:q!
```

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 6.12.74+deb13+1-amd64 x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
This lab has two user accounts (username :: password )
```

```
root      :: netlab123
sysadmin  :: netlab123
```

```
Press the [Enter] key to begin...
```

```
sysadmin@localhost:~$ vi myfile
sysadmin@localhost:~$
```

## 11.2.50 Step 50

Open myfile with the `vi` editor:

```
vi myfile
```

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

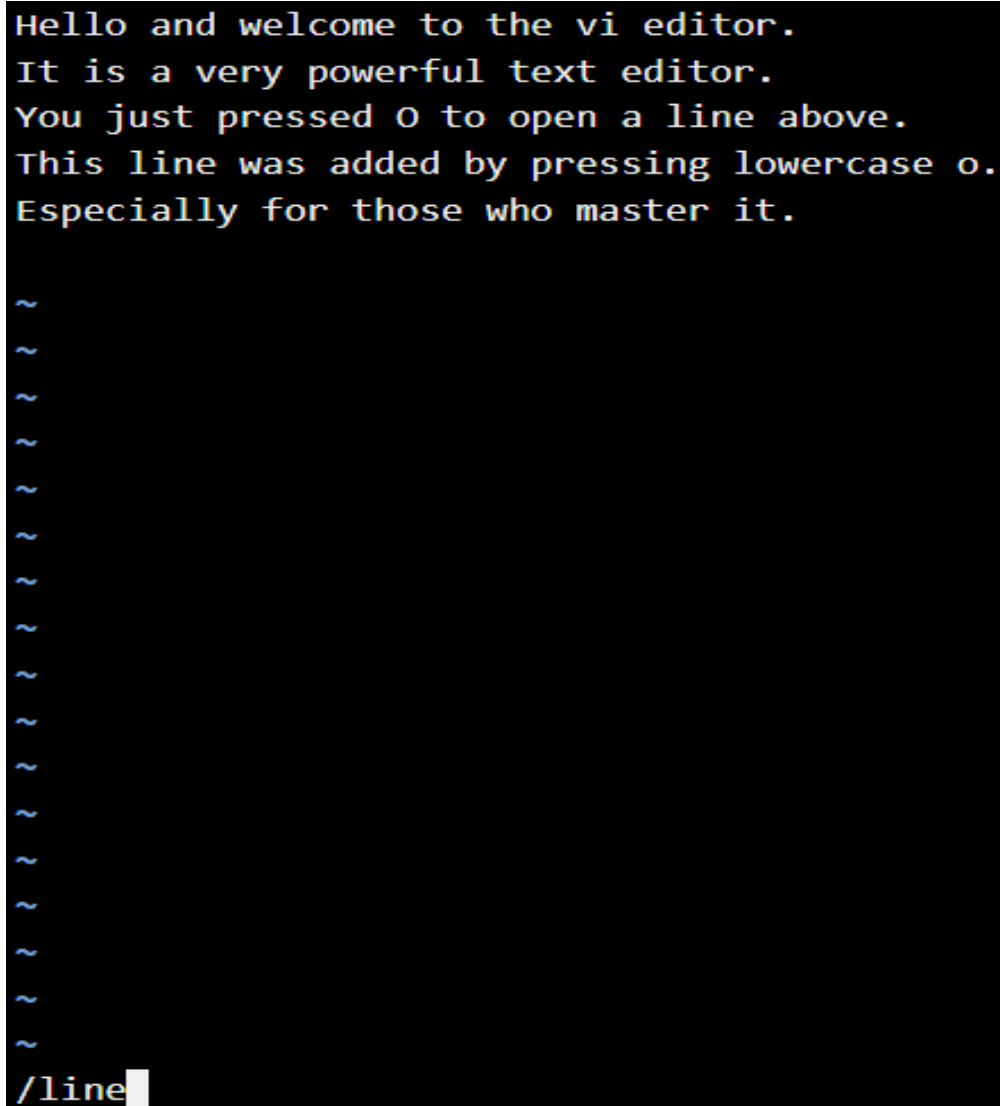
Notice that lines 3 and 4 are still present.

## 11.2.51 Step 51

Search forward for the word `line`. You'll notice the cursor moves to the beginning of the first instance of the word `line` as shown in image below:

```
/line
```

Your screen should look similar to the following:



The screenshot shows the vi editor interface with a black background and white text. The first five lines of the file contain the following text:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

Below these lines are several empty lines, each starting with a tilde (~) character. At the bottom of the screen, the search command `/line` is entered, and the cursor is positioned at the beginning of the first instance of the word `line` in the fifth line of the file.

## 11.2.52 Step 52

Search for the next instance of the word `line` by pressing the letter `n`:

n

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

~~~~~

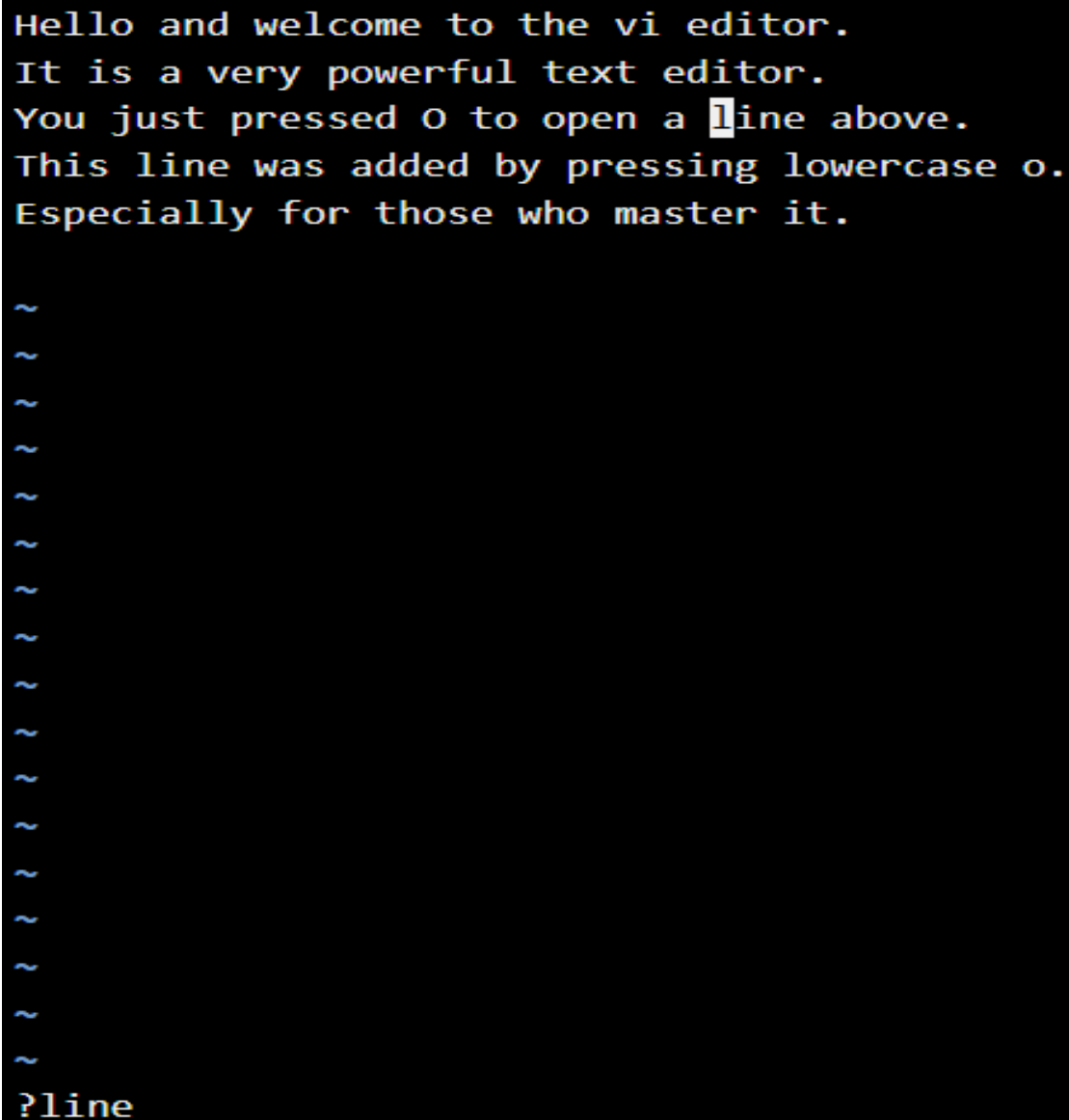
/line

11.2.53 Step 53

Search backward for the word `line`. You'll notice the cursor moves to the beginning of the previous instance of the word `line` as shown in image below:

```
?line
```

Your screen should look similar to the following:



The screenshot shows the vi editor interface with a black background and yellow text. The first five lines of text are: "Hello and welcome to the vi editor.", "It is a very powerful text editor.", "You just pressed O to open a **l**ine above.", "This line was added by pressing lowercase o.", and "Especially for those who master it.". Below these lines are several lines of tilde (~) characters. At the bottom left, the search command "?line" is entered, and the cursor is positioned at the start of the word "line" in the third line of text.

11.2.54 Step 54

Search for the previous instance of the word `line` by pressing the letter `n`. Since there are none in this direction, `vi` will wrap around the document:

n

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This line was added by pressing lowercase o.  
Especially for those who master it.
```

~~~~~

search hit TOP, continuing at BOTTOM

## 11.2.55 Step 55

You will replace the word `line` with the word `entry`. When you press `cw` you will be in insert mode and you will be able to type over the word `line`:

```
cw  
entry
```

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This entry Was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.56 Step 56

Press **Esc** key to exit insert mode.

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This entry was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

## 11.2.57 Step 57

Add text at the beginning of a line. Enter insert mode again and add a line by pressing upper case **i**:

I

Your screen should look similar to the following:

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This entry was added by pressing lowercase o.  
Especially for those who master it.
```

~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~

Insert modes include: **i**, **I**, **a**, **A**, **o**, and **O**

## 11.2.58 Step 58

Press the **Esc** key to return to command mode.

```
Hello and welcome to the vi editor.  
It is a very powerful text editor.  
You just pressed O to open a line above.  
This entry was added by pressing lowercase o.  
Especially for those who master it.
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

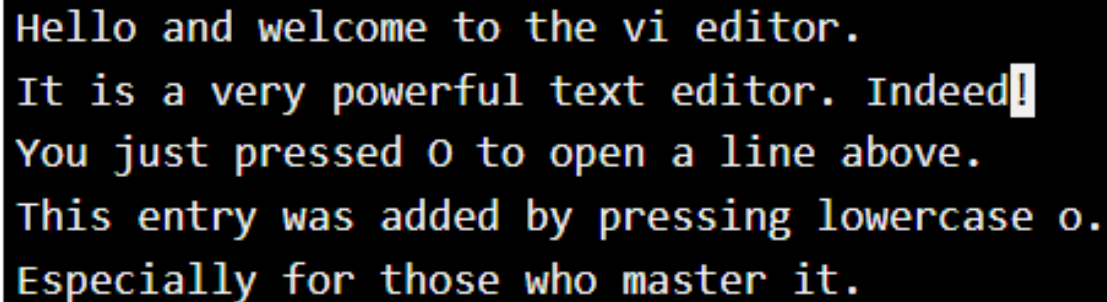
## 11.2.59 Step 59

Add text at the end of a line (uppercase **A**). First move to the second line and add the phrase `Indeed!`:

```
2G
A
[Space] Indeed!
```

Press the **Esc** key to return to command mode.

Your screen should look similar to the following:



```
Hello and welcome to the vi editor.
It is a very powerful text editor. Indeed!
You just pressed O to open a line above.
This entry was added by pressing lowercase o.
Especially for those who master it.
```

```
~
~
~
~
~
~
~
```

## 11.2.60 Step 60

Save your changes and exit `vi`:

```
:x
```

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 6.12.74+deb13+1-amd64 x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
This lab has two user accounts (username :: password )
```

```
root      :: netlab123
sysadmin  :: netlab123
```

```
Press the [Enter] key to begin...
```

```
sysadmin@localhost:~$ vi myfile
sysadmin@localhost:~$ vi myfile
sysadmin@localhost:~$
```



## 11.3 Basic Shell Scripting

Shell scripting allows you to take a complex sequence of commands, place them into a file and then run the file as a program. This saves you the time of having to repeatedly type a long sequence of commands that you routinely use.

This lab will focus on how to create simple shell scripts. For the purpose of this lab, it is assumed that you know how to use a text editor. Feel free to use the editor of your choice: `vi`, `nano`, `gedit` or any other editor that you like.

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 6.12.74+deb13+1-amd64 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

This lab has two user accounts (username :: password )

    root      :: netlab123
    sysadmin  :: netlab123

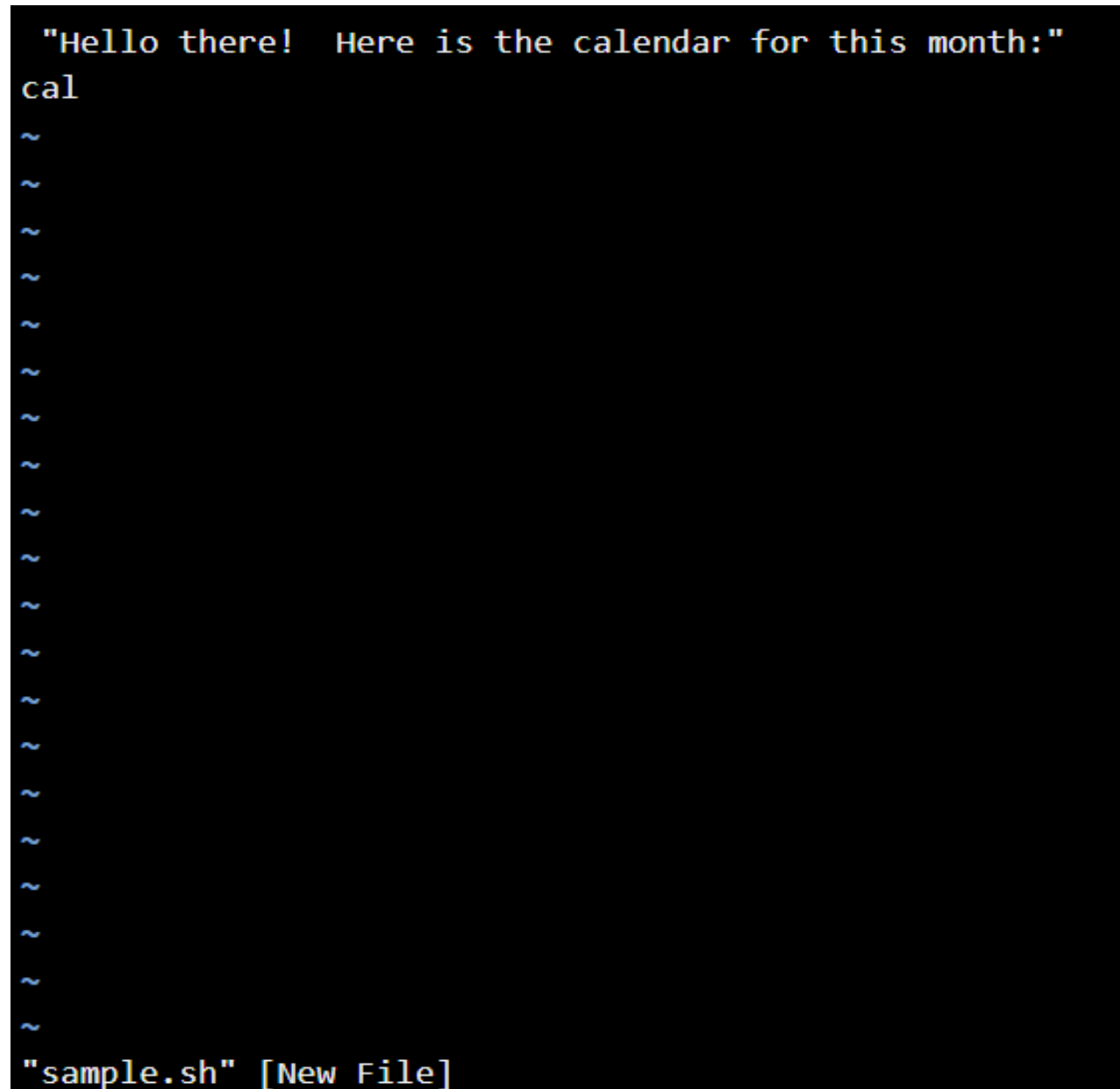
Press the [Enter] key to begin...

sysadmin@localhost:~$ vi myfile
sysadmin@localhost:~$ vi myfile
sysadmin@localhost:~$
```

## 11.3.1 Step 1

To create a simple shell script, you just need to create a text file and add commands. Create a file called `sample.sh` and add the following lines:

```
echo "Hello there!  Here is the calendar for this month:"  
cal
```



A screenshot of a terminal window with a black background. The first line shows the command `"Hello there! Here is the calendar for this month:"` in a yellow monospace font. The second line shows the command `cal` in the same font. Below these, there are 18 lines, each starting with a blue tilde `~` character. At the bottom of the terminal, the text `"sample.sh" [New File]` is displayed in the same yellow monospace font.

## 11.3.2 Step 2

To make it clear that this is a BASH shell script, you need to include a special line at the top of the file called a "shbang" (or "shebang"). This line starts with `#!` and then contains the path to the BASH shell executable.

Add the following line at the top of the `sample.sh` file:

```
#!/bin/bash
```

```
#!/bin/bash
"Hello there! Here is the calendar for this month:"
cal
~
~
~
~
~
~
~
~
~
~
~
~
```

## 11.3.3 Step 3

One way that you can run this program is by typing `bash` before the filename. Execute the following:

```
bash sample.sh
```

```
sysadmin@localhost:~$ vi sample.sh
sysadmin@localhost:~$ bash sample.sh
Hello there! Here is the calendar for this month:
    April 2026
Su Mo Tu We Th Fr Sa
           1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

sysadmin@localhost:~$
```

## 11.3.4 Step 4

You can avoid having to type `bash` in front of the filename by making the file "executable" for all users. Run the following commands:

```
ls -l sample.sh
chmod a+x sample.sh
ls -l sample.sh
./sample.sh
```

```
sysadmin@localhost:~$ ls -l sample.sh
-rw-rw-r-- 1 sysadmin sysadmin 75 Apr  2 19:07 sample.sh
sysadmin@localhost:~$ chmod a+x sample.sh
sysadmin@localhost:~$ ls -l sample.sh
-rwxrwxr-x 1 sysadmin sysadmin 75 Apr  2 19:07 sample.sh
sysadmin@localhost:~$ ./sample.sh
Hello there! Here is the calendar for this month:

    April 2026
Su Mo Tu We Th Fr Sa
                1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

sysadmin@localhost:~$
```

The `chmod` command is used to change permissions on the file so that the file can be executed.

## 11.3.5 Step 5

A common feature used in scripting is "backquoting". With this technique, you can run a shell command "within" another shell command. The outcome of the internal command will be returned as an argument to the external command.

Add the following to the bottom of the `sample.sh` file:

```
vi sample.sh
```

```
sysadmin@localhost:~$ vi sample.sh
```

```
echo "Today is" `date +%A`
```

```
#!/bin/bash
echo "Hello there! Here is the calendar for this month:"
cal
echo "Today is" `date +%A`
~
~
```

Exit insert mode by pressing **Esc**, then type `:wq!` and hit **Enter** to save and exit the file.

```
cat sample.sh
./sample.sh
```

```
sysadmin@localhost:~$ cat sample.sh
#!/bin/bash
echo "Hello there! Here is the calendar for this month:"
cal
echo "Today is" `date +%A`
sysadmin@localhost:~$ ./sample.sh
Hello there! Here is the calendar for this month:
    April 2026
Su Mo Tu We Th Fr Sa
           1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

Today is Thursday
sysadmin@localhost:~$
```

## 11.3.6 Step 6

You have been using `./` in front of the `sample.sh` filename to indicate that the file is in the current directory. Execute the following to see how the shell would fail to find the file if you don't use the `./`:

```
sample.sh
```

Your screen should look like the following:

```
Hello there!  Here is the calendar for this month:
```

```
    April 2026
```

```
Su Mo Tu We Th Fr Sa
```

```
      1 2  3  4
```

```
  5  6  7  8  9 10 11
```

```
12 13 14 15 16 17 18
```

```
19 20 21 22 23 24 25
```

```
26 27 28 29 30
```

```
Today is Thursday
```

```
sysadmin@localhost:~$ sample.sh
```

```
sample.sh: command not found
```

```
sysadmin@localhost:~$
```

## 11.3.7 Step 7

Recall that the `$PATH` variable is used to search for commands that you type. Execute the following to see the `$PATH` variable for the `sysadmin` account:

```
echo $PATH
```

```
sysadmin@localhost:~$ ./sample.sh
Hello there! Here is the calendar for this month:
    April 2026
Su Mo Tu We Th Fr Sa
                1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

Today is Thursday
sysadmin@localhost:~$ sample.sh
sample.sh: command not found
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games:/snap/bin
sysadmin@localhost:~$
```



## 11.3.8 Step 8

Note that `/home/sysadmin/bin` is one of the directories in the `$PATH` variable. This is a great place to put your shell scripts:

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games:/snap/bin
sysadmin@localhost:~$ mkdir bin
sysadmin@localhost:~$ mv sample.sh bin
sysadmin@localhost:~$ sample.sh
Hello there! Here is the calendar for this month:
    April 2026
Su Mo Tu We Th Fr Sa
           1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

Today is Thursday
sysadmin@localhost:~$
```

## 11.4 Conditional and Repetitive Execution

Note that in this section examples that are more complex will be demonstrated. When doing so, you will be using a technique to describe what is happening in the program. The technique will look like the following:

| Enter this column into drive.sh           | This column describes the code (don't enter into the file) |
|-------------------------------------------|------------------------------------------------------------|
| <code>echo "Please enter your age"</code> | <code># print a prompt</code>                              |
| <code>read age</code>                     | <code># read user input and place in \$age variable</code> |

When following the instructions provided, you are to enter the text from the left column into the specified file (`drive.sh` in the example above). The right column is used to describe specific lines in the program. The pound (hash) sign `#` character is used because in a shell script you can place comments within your program by using a `#` character.

# 11.4.1 Step 1

Scripts that are more complex may make use of conditional execution. A conditional expression, like the `if` statement, can make use of the outcome of a command called `test`. The `test` statement compares two numbers (or two strings) for things like "equal to", "less than", etc.

Create the following `drive.sh` file and make it executable to see how the `if` and `test` statements work.

```
vi drive.sh
```

Begin by placing the following in `drive.sh`:

| Enter this column into drive.sh                          | This column describes the code (do not enter into the file)                          |
|----------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>#!/bin/bash</code>                                 |                                                                                      |
| <code>echo "Please enter your age"</code>                | <code># print a prompt</code>                                                        |
| <code>read age</code>                                    | <code># read user input and place in the \$age variable</code>                       |
| <code>if test \$age -lt 16</code>                        | <code># test \$age -lt 16 returns "true" if \$age is numerically less than 16</code> |
| <code>then</code>                                        |                                                                                      |
| <code>    echo "You are not old enough to drive."</code> | <code># executes when test is true</code>                                            |
| <code>else</code>                                        |                                                                                      |
| <code>    echo "You can drive!"</code>                   | <code># executes when test is false</code>                                           |
| <code>fi</code>                                          | <code># This ends the if statement</code>                                            |

Then make the file executable and run it:

```
cat drive.sh
chmod a+x drive.sh
./drive.sh
```

Your screen should look like the following:

```
sysadmin@localhost:~$ cat drive.sh
#!/bin/bash
echo "Please enter your age"
read age
if test $age -lt 16
then
    echo "You are not old enough to drive."
else
    echo "You can drive!"
fi
sysadmin@localhost:~$ chmod a+x drive.sh
sysadmin@localhost:~$ ./drive.sh
Please enter your age
14
You are not old enough to drive.
sysadmin@localhost:~$
```

Verbally, you could read the `if` statement as *If \$age is less than 16, then echo 'You are not old enough to drive', else echo 'You can drive!'.* The `fi` ends the `if` statement.

## 11.4.2 Step 2

The test statement is automatically called when you place its arguments within square brackets `[ ]` surrounded by spaces. Modify the `if` line of `drive.sh` so it looks like the following:

```
if [ $age -lt 16 ]
```

Then run the program again:

```
cat drive.sh
./drive.sh
```

Your screen should look like the following:

```
sysadmin@localhost:~$ cat drive.sh
#!/bin/bash
echo "Please enter your age"
read age
if [ $age -lt 16 ]
then
    echo "You are not old enough to drive."
else
    echo "You can drive!"
fi
sysadmin@localhost:~$ ./drive.sh
Please enter your age
21
You can drive!
sysadmin@localhost:~$
```

To see a full list of test conditions, run the command `man test`.

### Important

There must be spaces around the square brackets. `[$age -lt 16]` would fail, but `[ $age -lt 16 ]` would work.

## 11.4.3 Step 3

You can also use the outcome of other shell commands as they all return "success" or "failure". For example, create and run the following program, which can be used to determine if a user account is on this system.

create a file called `check.sh`:

```
vi check.sh
```

```
sysadmin@localhost:~$ vi check.sh
```

Add the following to `check.sh`:

```
#!/bin/bash
echo "Enter a username to check: "
read name
if grep $name /etc/passwd > /dev/null
then
    echo "$name is on this system"
else
    echo "$name does not exist"
fi
```

Then run the following commands:

```
cat check.sh
chmod a+x check.sh
./check.sh
```

When prompted for a username, give the value of "root". Execute the command again (`./check.sh`) and provide the value of "bobby". Your screen should look like the following:

```
sysadmin@localhost:~$ vi check.sh
sysadmin@localhost:~$ cat check.sh
#!/bin/bash
echo "Enter a username to check: "
read name
if grep $name /etc/passwd > /dev/null
then
echo "$name is on this system"
else
echo "$name does not exist"
fi
sysadmin@localhost:~$ chmod a+x check.sh
sysadmin@localhost:~$ ./check.sh
Enter a username to check:
root
root is on this system
sysadmin@localhost:~$ ./check.sh
Enter a username to check:
bobby
bobby does not exist
sysadmin@localhost:~$
```

## 11.4.4 Step 4

Another common conditional statement is called the `while` loop. This statement is used to execute code repeatedly as long as a conditional check returns "true". Begin by creating a file called `num.sh`:

```
vi num.sh
```

Enter this into the *num.sh* file

This describes the code (don't enter into the file)

```
#!/bin/bash

echo "Please enter a number greater than
100"

read num

while [ $num -le 100 ]

do                                     # Execute code from do to done if test condition is
                                     "true"

    echo "$num is NOT greater than 100."

    echo "Please enter a number greater than
100"

    read num

done                                  # This ends the while statement

echo "Finally, $num is greater than 100"
```

Then make the file executable and run it:

```
cat num.sh
chmod a+x num.sh
./num.sh
```

When prompted for a number, enter 25. When prompted again, enter 99. Finally, enter 101 when prompted for a number the third time. Your screen should look like the following:

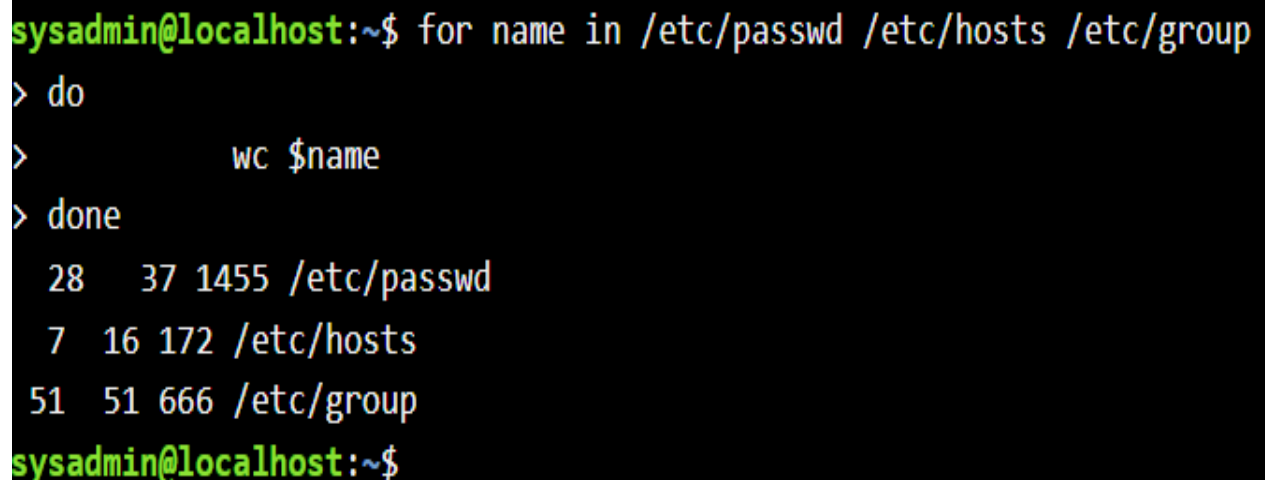
```
sysadmin@localhost:~$ cat num.sh
#!/bin/bash
echo "Please enter a number greater than 100"
read num
while [ $num -le 100 ]
do
echo "$num is NOT greater than 100."
echo "Please enter a number greater than 100"
    read num
done
echo "Finally, $num is greater than 100"
sysadmin@localhost:~$ chmod a+x num.sh
sysadmin@localhost:~$ ./num.sh
Please enter a number greater than 100
25
25 is NOT greater than 100.
Please enter a number greater than 100
99
99 is NOT greater than 100.
Please enter a number greater than 100
101
Finally, 101 is greater than 100
sysadmin@localhost:~$
```

## 11.4.5 Step 5

Scripting code is part of the BASH shell, which means you can use these statements on the command line just like you use them in a shell script. This can be useful for a statement like the `for` statement, a statement that will assign a list of values one at a time to a variable. This allows you to perform a set of operations on each value. For example, run the following on the command line:

```
for name in /etc/passwd /etc/hosts /etc/group
do
    wc $name
done
```

Your screen should look like the following:



```
sysadmin@localhost:~$ for name in /etc/passwd /etc/hosts /etc/group
> do
>     wc $name
> done
 28  37 1455 /etc/passwd
  7  16  172 /etc/hosts
 51  51  666 /etc/group
sysadmin@localhost:~$
```

Note that the `wc` command was run three times: once for `/etc/passwd`, once for `/etc/hosts` and once for `/etc/group`.

## 11.4.6 Step 6

Often the `seq` command is used in conjunction with the `for` statement. The `seq` command can generate a list of integer values, for instance from 1 to 10. For example, run the following on the command line to create 12 files named `test1`, `test2`, `test3`, etc. (up to `test12`):

```
ls
for num in `seq 1 12`
do
    touch test$num
done
ls
```

```
sysadmin@localhost:~$ ls
Desktop  Downloads  Pictures  Templates  bin        drive.sh  num.sh
Documents Music      Public    Videos     check.sh  myfile

sysadmin@localhost:~$ for num in `seq 1 12`
> do
>     touch test$num
> done
sysadmin@localhost:~$ ls
Desktop  Music  Templates  check.sh  num.sh  test11  test3  test6  test9
Documents Pictures Videos    drive.sh  test1   test12  test4  test7
Downloads Public    bin       myfile    test10  test2   test5  test8

sysadmin@localhost:~$
```